Physics-informed neural networks (v1.0)

Rasmus Hag Løvstad, rhalo@dtu.dk
Johanna Marie Gegenfurtner, johge@dtu.dk
Clara E. A. Edmonds, claed@dtu.dk

October 2026

Introduction

Differential equations are a common modeling tool across many different fields in both technical and social sciences [Cai et al., 2021, Navarro et al., 2023]. While some can be solved in closed form, many require compute-expensive solvers, such as 3D finite element simulations or computational fluid dynamics simulation. This project pertains to training a neural network to approximate differential equations with respect to specific initial conditions and constraints.
Pick this project if: you want learn more about scientific machine learning, you are curious about how physical knowledge can make neural networks smarter, and you enjoy seeing mathematics connect directly to code.
Key concepts: scientific machine learning, ordinary differential equations (ODEs), learning with constraints.

Learning Outcomes

At the end of this project, you will be able to:

- Derive first and second order derivatives of a neural network with respect to its inputs analytically, and connect these to the chain rule and backpropagation
- Implement a physics-informed loss function from scratch, including residual and initial condition terms, without using autograd
- Explain the difference between soft and hard enforcement of physical constraints and the practical consequences of each
- Train a neural network on scarce data using physical equations as a regulariser, and critically evaluate when this helps and when it does not
- Compare hand-derived and autograd-based gradient implementations quantitatively and reflect on what each reveals about the network's structure

Note: We highly recommend solving the following tasks sequentially, since later tasks require intuition which is gathered during the first tasks. **The use of any AI tools for coding or writing your project report is strictly prohibited.** If you have questions, please ask us during the exercise sessions. We will try to reply as soon as we can. In particular, if you struggle with any

mathematical parts of the assignment, we are happy to check your solutions. Please include any lengthy computations in the appendix of your project report. In general, we recommend concise writing, and we value interesting insights over verbosity. You do not need to write an introduction to your report, and can start with your reporting your answers to Task 1.

Background

A standard neural network learns from data alone — given enough examples, it finds a function that fits them. But what if you have very few data points, or you already know the physical law governing the system? Physics-informed neural networks (PINNs) answer this question by encoding the governing equation directly into the loss function. Rather than only penalising deviation from data, the network is also penalised for violating the physics at a set of collocation points — points sampled from the domain where the equation should hold. The network must therefore satisfy two objectives simultaneously: fit the data, and obey the physics.

Physics-informed neural networks (PINNs) embed information about the underlying physics of a system into the neural networks. This is done by adding additional terms to the loss that penalize predictions which do not respect the physics equations. Collocation points are points in the domain at which the physics residual is evaluated during training. They are distinct from data points — no labels are needed at collocation points, only the ability to evaluate the equation. In practice, collocation points are sampled densely across the domain to ensure the physics is enforced everywhere, not just where data is available.

Physics-informed neural networks are especially useful in the following cases:

- Little to no data is available. Since PINNs have additional information in the form of the underlying equations, they perform better than vanilla neural networks when few data points are available.
- More realistic, physical predictions. Since PINNs are encouraged to predict values which respect the differential equations, these values are less likely to be completely unrealistic. This also applies to out-of-distribution generalization on unseen data, a topic where regular neural networks often struggle.
- Certain parameters in the physics equations are unknown. PINNs can be used to solve inverse problems, such as specific parameter values in the differential equations. This will not be covered in this project.

Physics-informed neural networks were first introduced by Raissi et al. [2019].

Task 1: Manual derivation of core derivatives

In week 1, in the notebook `1.1 FNN Pen and Paper.ipynb`, you derived the equations for an output y for a shallow neural network, writing y in terms of the weights w and inputs x. For this task, you will derive similar derivatives, which are core to differential equations.

Task 1.1 Derive $\frac{\partial y}{\partial x}$ and $\frac{\partial^2 y}{\partial^2 x}$ by hand analytically for a feedforward network with 1 hidden layer and tanh activations, expressing the result directly in terms of the weight matrices $W^{(l)}$ and biases $b^{(l)}$.

Task 1.2 Discuss compute costs of second-order derivatives versus first-order derivatives and how PINNs might/might not scale depending on the derivatives present.

Task 2: Damped harmonic oscillator

For this task, you will implement a PINN that will solve a simple damped harmonic oscillator ODE.

Background

In the following, we summarise the most important facts you will need for this problem. For further reading we recommend Feynman [1963]. When the mass is displaced from its equilibrium point (we define this as $x = 0$), the mass will oscillate periodically around this point. If the oscillations are damped, which we assume they are, the oscillations will cease and the mass will once again rest at its equilibrium point $x = 0$.

The partial differential equation which governs the behaviour of the harmonic oscillator is:

$$m\frac{d^2x}{dt^2} + \mu\frac{dx}{dt} + kx = 0,$$

where m is the mass of the object at the end of the spring, μ is the friction constant and k is the spring constant, x indicates the displacement of the mass away from the equilibrium position, and t is the time variable. Dividing by m, we may rewrite this as follows:

$$\frac{d^2x}{dt^2} + \frac{\mu}{m}\frac{dx}{dt} + \frac{k}{m}x = 0. \tag{1}$$

Without loss of generality, we consider the case $m = 1$ (otherwise we may just adapt our time unit). We impose the following initial conditions: at time $t = 0$, the mass is displaced from its equilibrium point, and the velocity of the mass is zero:

$$x(0) = 1 \text{ and } \frac{dx}{dt}(0) = 0. \tag{2}$$

Depending on these initial conditions we can compute the trajectory of $x(t)$ on an interval $t \in [0, T]$. We define two new variables to simplify the solution. These new variables are

$$\delta = \frac{\mu}{2m}, \omega_0 = \sqrt{k} \text{ and } \omega = \sqrt{\omega_0^2 - \delta^2}.$$

The closed form solution can be expressed as

$$x(t) = e^{-\delta t}(2A\cos(\phi + \omega t)), \tag{3}$$

with

$$A = \frac{1}{2\cos\phi} \text{ and } \phi = \tan^{-1}(\frac{\delta}{\omega}).$$

Task 2.1 Generate 10 data points for $t \in (0, 0.5)$ using the above closed form solution in eq. 3. Note that these data points are distinct from the collocation points used to enforce the physics loss.

Task 2.2 Fit a simple feedforward neural network with 1 hidden layer, tanh as your activation function, and mean squared error as the loss to these 10 data points. Infer on the time domain (0, 1). Observe that the neural network approximation deteriorates on the interval $t \in (0.5, 1.0)$. Explain in your own words why this occurs — what property of the training data causes the network to fail here, and what information would be needed to correct it?

Task 2.3 Design a loss function that regularises the network using the physics of the system. Your loss should balance three objectives — fitting the data, satisfying Equation 1 at a set of collocation points, and enforcing the initial conditions in Equation 2:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_{\text{phys}} \mathcal{L}_{\text{phys}} + \lambda_{\text{IC}} \mathcal{L}_{\text{IC}}$$

Write out $\mathcal{L}_{\text{phys}}$ and $\mathcal{L}_{\text{IC}}$ explicitly in terms of the network weights $W^{(l)}$ and biases $b^{(l)}$, using the derivative expressions from Task 1. Then derive the parameter gradients $\frac{\partial \mathcal{L}}{\partial W^{(1)}}$ and $\frac{\partial \mathcal{L}}{\partial W^{(2)}}$ by applying the chain rule through your analytical expressions — do not use autograd.

Task 2.4 Before training, verify your analytical gradients from Task 2.3 against numerical finite differences. For a randomly initialised network, compute both the analytical gradient $\frac{\partial \mathcal{L}}{\partial W_{ij}^{(1)}}$ and the finite difference approximation:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(1)}} \approx \frac{\mathcal{L}(W_{ij}^{(1)} + \epsilon) - \mathcal{L}(W_{ij}^{(1)} - \epsilon)}{2\epsilon}$$

using $\epsilon = 10^{-5}$.

Task 2.5 Train a neural network by implementing the parameter gradients $\frac{\partial \mathcal{L}}{\partial W^{(1)}}$ and $\frac{\partial \mathcal{L}}{\partial W^{(2)}}$ analytically using the expressions derived in the previous task — do not use autograd for this step. Show the resulting losses, describe what weights λ you assign to each term and explain your reasoning. Plot inference results on the interval $t \in (0, 1.0)$ for weak and strong regularization.

Task 2.6 Repeat the training from the previous task using `torch.autograd.grad` to compute the parameter gradients. Compare the results against your hand-derived implementation: report the relative error between the two sets of gradients at initialization, and verify that both implementations produce equivalent loss curves and inference results. Discuss any differences you observe in training behavior or final performance, and reflect on what the hand-derived implementation revealed about the gradient structure that autograd obscures.

Task 2.7 In the previous tasks, the initial conditions were enforced as a soft penalty via $\mathcal{L}_{\text{IC}}$. An alternative is to build the initial conditions directly into the network architecture, so that they are satisfied exactly for any choice of weights — this is called *hard enforcement*. Consider the modified network output:

$$\tilde{x}_\theta(t) = 1 + t^2 \cdot x_\theta(t)$$

Show analytically that this construction satisfies both initial conditions in Equation 2 for any choice of network weights. Explain why t^2 is needed rather than t. Then remove $\mathcal{L}_{\text{IC}}$ from your loss function, substitute $\tilde{x}_\theta$ for x_θ throughout, and retrain the network. Compare your results against Task 2.5 on the following: does the solution satisfy the initial conditions more precisely? Does removing $\mathcal{L}_{\text{IC}}$ change the training dynamics or the final inference quality on $t \in (0.5, 1.0)$? Reflect on when you would prefer hard over soft enforcement, and whether there are situations where soft enforcement might be preferable.

Task 3: Build your own PINN

For this task, you will choose an ODE yourself and build an appropriate PINN. Please pick **one** of the following three ODEs or bring your own.

The **2D Laplace equation**. The 2D Laplace equation describes flow. In this case we will consider potential flow of an incompressible liquid, with no vorticity, around a cylinder of radius R. Far away from the cylinder, the flow is uniform with magnitude U. The governing equation in Cartesian coordinates is $\nabla^2\phi = \frac{\partial^2\phi}{\partial x^2} + \frac{\partial^2\phi}{\partial y^2} = 0$. The exact solution in polar coordinates is given by:

$$\phi(r,\theta) = Ur(1 + \frac{R^2}{r^2})\cos\theta$$

$$V_r = \frac{\partial\phi}{\partial r} = U(1 - \frac{R^2}{r^2})\cos\theta$$

$$V_\theta = \frac{1}{r}\frac{\partial\phi}{\partial\theta} = -U(1 + \frac{R^2}{r^2})\sin\theta$$

Please use the following conditions: flow goes from left to right, inlet condition at the left hand side $V_x = 0.1$, outlet condition at the right hand side $\phi = 1.0$. There is no flux allowed into the cylinder or leaving the upper or lower walls. See Wikipedia [Wikipedia, 2026] or this blog post [le Bourdais, 2020] for more information.

The **1D Euler-Bernoulli beam equation** describes how beams, such as (idealized) railway tracks, behave under loading and bending. For a cantilever beam of length L with a point load q (in N) applied at the free end $x = L$, the deflection $w(x)$ satisfies:

$$EI\frac{d^4w}{dx^4} = 0$$

where $EI = 10^{13}\,\mathrm{kN}\cdot\mathrm{m}^2$ is the flexural rigidity constant and $x \in [0, L]$. The beam is fixed at $x = 0$, giving the boundary conditions:

$$w(0) = 0, \quad \frac{dw}{dx}(0) = 0, \quad \frac{d^2w}{dx^2}(L) = 0, \quad EI\frac{d^3w}{dx^3}(L) = -q$$

The closed-form solution is:

$$w(x) = -\frac{qx^2}{6EI}(3L - x)$$

See MechaniCalc [2026] for more information.

The **1D heat equation** describes how heat diffuses along a rod over time. The temperature $u(x,t)$ at position x and time t satisfies:

$$\frac{\partial u}{\partial t} = \alpha\frac{\partial^2 u}{\partial x^2}, \quad x \in [0, L], \quad t \in [0, T]$$

where $\alpha > 0$ is the thermal diffusivity constant. We hold both ends of the rod at zero temperature for all time:

$$u(0,t) = 0, \quad u(L,t) = 0$$

and impose the initial condition:

$$u(x,0) = \sin\Big(\frac{\pi x}{L}\Big)$$

The closed-form solution is:

$$u(x,t) = \sin\Big(\frac{\pi x}{L}\Big)\exp\left(-\alpha\left(\frac{\pi}{L}\right)^2 t\right)$$

Bring your own equation You can also bring your own ODE or PDE as long as you have talked to us about it first.

Task 3.1 Visualize the ODE in a suitable format. Plot 5 different trajectories based on different starting positions using a closed-form solution, if available. If no closed-form solution is available for your ODE (if you chose to use your own), feel free to use a solver.

Task 3.2 Derive the residual loss and all constraint losses necessary to train a PINN.

Task 3.3 Sample from a suitable grid and train a PINN of suitable architecture. Plot loss curves and L_2-error compared to the closed-form solution. Show your solutions for the same initial starting positions as Task 3.1 and compare.

Task 3.4 Reflect on your solution. Is your PINN sufficiently accurate compared to the closed-form solution? If not, how would you improve it? When, if ever, do you think that PINNs offer a viable alternative to ODE solvers?

References

Shengze Cai, Zhiping Mao, Zhicheng Wang, Minglang Yin, and George Em Karniadakis. Physics-informed neural networks (pinns) for fluid mechanics: A review, 2021. URL https://arxiv.org/abs/2105.09506.

Richard Phillips Feynman. The feynman lectures on physics. lecture 21: The harmonic oscillator. *The Feynman lectures on physics*, 1:46, 1963. URL https://www.feynmanlectures.caltech.edu/I_21.html.

Frolian le Bourdais. Simulating 2D potential flow using finite differences in Python | Frolian's blog, 2020. URL https://flothesof.github.io/2D-potential-flow-finite-differences.html.

MechaniCalc. Beam Deflection Tables | MechaniCalc, 2026. URL https://mechanicalc.com/reference/beam-deflection-tables.

Luis Medrano Navarro, Luis Martín Moreno, and Sergio G Rodrigo. Solving differential equations with deep learning: a beginner's guide, 2023. URL https://arxiv.org/abs/2307.11237.

M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, February 2019. ISSN 0021-9991. doi: 10.1016/j.jcp.2018.10.045. URL https://www.sciencedirect.com/science/article/pii/S0021999118307125.

Wikipedia. Potential flow around a circular cylinder, March 2026. URL https://en.wikipedia.org/w/index.php?title=Potential_flow_around_a_circular_cylinder&oldid=1344732932. Page Version ID: 1344732932.